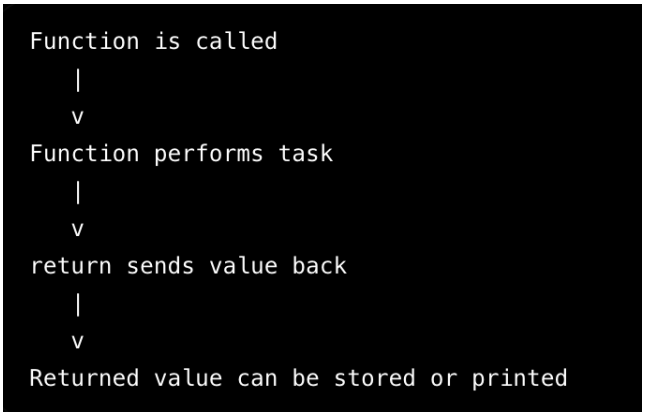
Flow Chart



Example 1

PYTHON CODE

```
def add(a, b):
    result = a + b
    return result

answer = add(10, 20)
print(answer)
```

Output:

30

Example 2: Function without return

PYTHON CODE

```
def greet():
    print("Hello")

result = greet()
print(result)
```

Output: Hello
None

Because greet() does not return any value, Python returns None.

deWithNishchal

10.8 Returning Multiple Values

Python functions can return multiple values.

Syntax:

```python
def function_name():
    return value1, value2
```

1. Multiple values can be returned using commas.
2. Python returns them as a tuple.
3. Returned values can be unpacked into variables.

Example 1

PYTHON CODE

```
def calculate(a, b):
    total = a + b
    difference = a - b
    return total, difference
sum_result, diff_result = calculate(20, 10)
print(sum_result)
print(diff_result)
```

Output:

30
10

10.9 Default Parameters

A default parameter has a predefined value. If no argument is passed, the default value is used.

Syntax

```python
def function_name(parameter=value):
    statement
```

1. Default parameters make arguments optional.
2. If an argument is provided, Python uses the given value.
3. If no argument is provided, Python uses the default value.
4. Non-default parameters must come before default parameters.

CodeWithNishchal

Example 1

PYTHON CODE

```
def greet(name="Guest"):
    print("Hello", name)
greet("Aman")
greet()
```

Output:

Hello Aman

Hello Guest

Correct and Incorrect Order

Code	Valid?	Reason
def show(name, age=18):	Yes	Default parameter comes after normal parameter
def show(name="Guest", age=18):	Yes	Both have defaults
def show(name="Guest", age):	No	Non-default parameter cannot come after default parameter

Important Warning: Avoid Mutable Default Values

Do not use mutable objects like list or dictionary as default values.

Wrong style:

```
def add_item(item, items=[]):
    items.append(item)
    return items

print(add_item("A"))
print(add_item("B"))
```

Output: ['A']

['A', 'B']

The same list is reused between function calls.

Better style:

```
Python

def add_item(item, items=None):
    if items is None:
        items = []

    items.append(item)
    return items

print(add_item("A"))
print(add_item("B"))
```

Output: ['A']

['B']

Coded with Kushal

10.10 Keyword Arguments

Keyword arguments pass values using parameter names.

Syntax

```
function_name(parameter_name=value)
```

1. Keyword arguments use names while calling a function.
2. Order does not matter when keyword arguments are used.
3. They make function calls more readable.
4. Positional arguments must come before keyword arguments.

Example 1

PYTHON CODE

```
def student_info(name, age, course):
    print("Name:", name)
    print("Age:", age)
    print("Course:", course)
student_info(age=21, course="Python", name="Aman")
```

Output: Name: Aman
Age: 21
Course: Python

Positional vs Keyword Arguments

Type	Example	Order Matters?
Positional argument	student_info("Aman", 21, "Python")	Yes
Keyword argument	student_info(age=21, name="Aman", course="Python")	No

Important Rule

Correct:

```
student_info("Aman", age=21, course="Python")
```

Incorrect:

```
student_info(name="Aman", 21, "Python")
```

Positional arguments cannot come after keyword arguments.

CodeWithNishchal

10.11 *args

*args is used when we do not know how many positional arguments will be passed.

Syntax

```
def function_name(*args):
    statement
```

1. *args collects extra positional arguments.
2. The collected values are stored as a tuple.
3. The name args is a convention; the * is important.
4. Use *args when the number of arguments is flexible.

Flow Chart

Function call has many positional arguments
  |
  v
*args collects them
  |
  v
Values are stored as a tuple

Example 1

PYTHON CODE

```
def add_numbers(*numbers):
    total = 0
    for number in numbers:
        total += number
    return total
print(add_numbers(10, 20, 30))
print(add_numbers(5, 15))
```

Output:
60
20
Here, numbers behaves like a tuple.

WithNishchal

10.12 **kwargs

**kwargs is used when we do not know how many keyword arguments will be passed.

Syntax

```
def function_name(**kwargs):
    statement
```

Explanation

1. **kwargs collects extra keyword arguments.
2. The collected values are stored as a dictionary.
3. Keys are argument names.
4. Values are argument values.
5. The name kwargs is a convention; the ** is important.

Flow Chart

Function call has many keyword arguments
  |
  v
**kwargs collects them
  |
  v
Values are stored as a dictionary

Example 1

PYTHON CODE

```
def show_profile(**details):
    for key, value in details.items():
        print(key, value)


show_profile(name="Aman", age=21,
course="Python")
```

Output:
name Aman
age 21
course Python

Here, details behaves like a dictionary.

CodeWithNishchal

10.13 *args vs **kwargs

Point	*args	**kwargs
Collects	Positional arguments	Keyword arguments
Stored as	Tuple	Dictionary
Symbol	Single star *	Double star **
Example call	func(10, 20, 30)	func(name="Aman", age=21)

Example 1

```
PYTHON CODE
def show_data(*args, **kwargs):
    print(args)
    print(kwargs)

show_data(10, 20, name="Aman", age=21)
```

Output:
(10, 20)
{'name': 'Aman', 'age': 21}

10.14 Function Parameter Order

When using different types of parameters together, the order matters.

Syntax

```
def function_name(normal_parameter, default_parameter=value, *args, **kwargs):
    statement
```

Explanation

1. Normal parameters come first.
2. Default parameters come after normal parameters.
3. *args comes after normal/default parameters.
4. **kwargs comes last.
5. This order keeps function calls clear and valid.

CodeWithNishchal

Parameter Order Table

Order	Parameter Type	Example
1	Normal parameter	name
2	Default parameter	age=18
3	*args	*marks
4	**kwargs	**details

Example 1

```
def student_report(name, age=18, *marks, **details):
    print("Name:", name)
    print("Age:", age)
    print("Marks:", marks)
    print("Details:", details)

student_report("Aman", 21, 80, 90, 85, course="Python", city="Delhi")
```

Output:

Name: Aman
Age: 21
Marks: (80, 90, 85)
Details: {'course': 'Python', 'city': 'Delhi'}

10.15 Docstrings

A **docstring** is a string written inside a function to explain what the function does.

Syntax

```
def function_name():
    """Function description."""
    statement
```

1. A docstring is written using triple quotes.
2. It is usually written as the first statement inside a function.
3. It explains the purpose of the function.
4. It helps other programmers understand the function.
5. It can be viewed using help() or .__doc__.

CodeWithNishchal

Example 1

```
def add(a, b):
    """Return the sum of two numbers."""
    return a + b

print(add(10, 20))
print(add.__doc__)
```

Output: 30

Return the sum of two numbers.

Good Docstring Style

```
Python

def calculate_area(length, width):
    """
    Return the area of a rectangle.

    length: rectangle length
    width: rectangle width
    """
    return length * width
```

10.16 Function Naming Rules

Function names follow the same basic rules as variable names.

Syntax

```
def function_name():
    statement
```

1. Function names should be meaningful.
2. Use snake_case for function names.
3. Function names should usually describe an action.
4. Avoid using Python keywords as function names.
5. Avoid unclear names like x(), abc(), or test1().

CodeWithNishchal

Chapter 11. Advanced Functions

Advanced functions help us write more flexible, reusable, and compact code.

Before starting, remember:

In Python, functions are objects. This means:

1. A function can be stored in a variable.
2. A function can be passed as an argument.
3. A function can be returned from another function.
4. A function can be written inside another function.

11.1 Functions as First-Class Objects

Syntax

```
def function_name():
    statement

new_name = function_name
```

In Python, functions behave like normal objects. We can store a function in a variable and call it using that variable.

Flow Chart

Create function
  |
  v
Assign function to variable
  |
  v
Call function using new variable

Example 1

PYTHON CODE

```
def greet():
    print("Hello Python")
message = greet
message()
```

Output:
Hello Python
Here, message refers to the same function as greet.

CodeWithNishchal

11.2 Lambda Functions

A **lambda function** is a small anonymous function. Anonymous means it does not need a normal function name.

Syntax

lambda arguments: expression

1. Lambda functions are used for small one-line functions.
2. They can take any number of arguments.
3. They can contain only one expression.
4. They automatically return the result of the expression.
5. Lambda functions are commonly used with map(), filter(), and sorting.

Example 1

Normal function:

```python
def square(number):
    return number * number

print(square(5))
```

Output:
25
Same logic using lambda:

```python
square = lambda number: number * number
```

Output: 25

Example 2

```python
add = lambda a, b: a + b

print(add(10, 20))
```

Output:30

Lambda is best for simple logic. For complex logic, use normal def functions.

CodeWithNishchal

11.3 map()

map() applies a function to every item of an iterable.

Syntax

map(function, iterable)

Explanation

1. map() takes a function and an iterable.
2. It applies the function to each item.
3. In Python 3, map() returns a map object, which is an iterator.
4. To display all results at once, convert it using list().

Example 1

PYTHON CODE

```
numbers = [1, 2, 3, 4]
squares = map(lambda number: number * number, numbers)
print(list(squares))
```

Output: [1, 4, 9, 16]

Example 2

```
names = ["aman", "riya", "kabir"]
upper_names = map(str.upper, names)
print(list(upper_names))
```

Output: ['AMAN', 'RIYA', 'KABIR']

map() Table

Part	Meaning
lambda number: number * number	Function to apply
numbers	Iterable
map()	Applies function to every item
list()	Converts result into a list

CodeWithPanchal

11.4 filter()

filter() is used to select items from an iterable based on a condition.

Syntax

filter(function, iterable)

Explanation

1. filter() takes a function and an iterable.
2. The function must return True or False.
3. Items that return True are kept.
4. Items that return False are removed.
5. In Python 3, filter() returns a filter object, which is an iterator.

Flow Chart

```
Iterable values
  |
  v
filter() checks condition
  |
  ├── True  -> keep item
  |
  └── False -> remove item
  |
  v
Filtered result
```

Example 1

PYTHON CODE

```
numbers = [1, 2, 3, 4, 5, 6]
even_numbers = filter(lambda number: number % 2 == 0, numbers)
print(list(even_numbers))
```

Output: [2, 4, 6]

map() vs filter()

Point	map()	filter()
Purpose	Transforms every item	Selects matching items
Output length	Usually same as input	Can be smaller
Function returns	New value	True or False
Example use	Square every number	Keep even numbers

CodeWithNishchal

11.5 reduce()

reduce() combines all items of an iterable into a single final value. reduce() is not directly available like map() and filter(). It must be imported from functools.

Syntax

from functools import reduce

reduce(function, iterable)

Explanation

1. reduce() takes a function and an iterable.
2. It combines values step by step.
3. It returns one final result.
4. It is useful for cumulative calculations.
5. For simple addition, sum() is usually better.

Example 1

PYTHON CODE

```
from functools import reduce
numbers = [1, 2, 3, 4]
total = reduce(lambda a, b: a + b, numbers)
print(total)
```

Output: 10
Working idea:
1 + 2 = 3
3 + 3 = 6
6 + 4 = 10

map() vs filter() vs reduce()

Function	Purpose	Final Result
map()	Transform items	Iterator of transformed items
filter()	Select items	Iterator of selected items
reduce()	Combine items	Single value

Code with Nishchal

11.6 Recursion

Recursion means a function calling itself.

Syntax

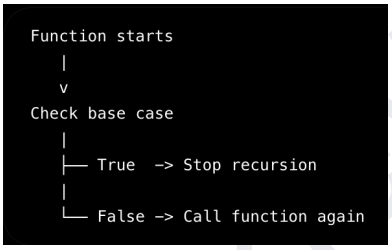
```python
def function_name():
    function_name()
```

A correct recursive function must have a stopping condition.

1. Recursion is used when a problem can be broken into smaller versions of the same problem.
2. A recursive function calls itself.
3. Every recursion must have a base case.
4. The base case stops the recursion.
5. Without a base case, recursion continues until Python raises RecursionError.

Part	Meaning
Base case	Condition that stops recursion
Recursive case	Function calling itself

Flow Chart



Example 1: Factorial

Factorial means: 5! = 5 × 4 × 3 × 2 × 1

PYTHON CODE

```python
def factorial(number):
    if number == 1:
        return 1
    return number * factorial(number - 1)
print(factorial(5))
```

Output: 120

factorial(5) = 5 * factorial(4) = 5 * 4 * factorial(3) = 5 * 4 * 3 * factorial(2) = 5 * 4 * 3 * 2 * factorial(1) = 5 * 4 * 3 * 2 * 1 = 120

CodeWithNishchal

11.7 Nested Functions

A nested function is a function defined inside another function.

Syntax

```
PYTHON CODE
def outer_function():
    def inner_function():
        statement

    inner_function()
```

1. A nested function is created inside another function.
2. The inner function can be used only inside the outer function.
3. Nested functions are useful for hiding helper logic.
4. They are also used in closures and decorators.

Example 1

```
PYTHON CODE
def outer():
    print("Outer function started")

    def inner():
        print("Inner function executed")

    inner()

outer()
```

Output: Outer function started
Inner function executed

Important point: inner() cannot be called directly outside outer().

11.8 Closures

A closure is created when an inner function remembers variables from its outer function, even after the outer function has finished.

Syntax

```
Python

def outer_function():
    variable = value

    def inner_function():
        use variable

    return inner_function
```

Code With Nishchal

1. A closure needs a nested function.
2. The inner function uses a variable from the outer function.
3. The outer function returns the inner function.
4. The inner function remembers the outer variable.
5. Closures are useful for creating customized functions.

```python
def multiplier(number):
    def multiply(value):
        return value * number

    return multiply

double = multiplier(2)

print(double(5))
```

Output: 10

11.9 Function Annotations

Function annotations are used to add type hints to function parameters and return values.

Syntax

def function_name(parameter: type) -> return_type:
    statement

1. Function annotations describe expected data types. They make code easier to understand.
2. They help editors and tools detect possible mistakes.
3. Python does not automatically enforce these types at runtime.
4. Annotations are stored in the function's __annotations__ attribute.

Example 1

PYTHON CODE

```
def add(a: int, b: int) -> int:
    return a + b
print(add(10, 20))
```

Output: 30

Python

```python
def add(a: int, b: int) -> int:
    return a + b

print(add.__annotations__)
```

Output: {'a': <class 'int'>, 'b': <class 'int'>, 'return': <class 'int'>}
Important point: Type hints are hints. Python does not stop wrong types automatically.

devWithNishchal

11.10 Higher-Order Functions

A higher-order function is a function that does at least one of these:

1. Takes another function as an argument.
2. Returns another function.

Syntax

```python
def outer():
    def inner():
        statement

    return inner
```

OR

```python
def outer(function):
    function()
```

Explanation

1. Higher-order functions are possible because Python functions are objects.
2. They are used in map(), filter(), reduce(), decorators, and callbacks.
3. They make code flexible and reusable.
4. They are common in functional programming.

Example 1: Function as Argument

PYTHON CODE

```python
def shout(text):
    return text.upper()

def process_text(function, text):
    return function(text)

result = process_text(shout, "hello")
print(result)
```

Output: HELLO

Here, process_text() is a higher-order function because it accepts another function.

Vitthal Nishchal